



Gems Seeker

Gems Seeker es un videojuego educativo y recreativo, diseñado para niños, cuyo objetivo es estimular la lógica, la atención y la estrategia mediante un entorno lúdico y amigable.

Autor(es):

Katherin Barrera

Nátaly Cárdenas

Valeria Florez

Laura Rivera

Género: Aventura / Puzzle educativo

Público objetivo: Niños y niñas en edad escolar

IDE: Visual Studio Code | Lenguaje: Python

Fecha: 25 de Septiembre, 2025

Materia: Estructura de Datos 2

UNIVERSIDAD DEL NORTE

The background is a dark blue gradient. It features several decorative elements: four large circles, each with a teal outer ring and a purple inner circle, positioned in the corners; and one smaller version of this design in the lower right area.

ÍNDICE

1. Introducción

2. Clases

3. Eventos

4. Videojuego

5. Conclusion

INTRODUCCIÓN

En un bosque 2D ancestral, el jugador debe recolectar gemas mágicas . Cada gema tiene un nivel de poder único (un número entero). El bosque está lleno de cofres ocultos que solo pueden abrirse si el jugador tiene la gema correcta, y la ubicación de cada gema se gestiona internamente mediante un Árbol Binario de Búsqueda.

OBJETIVOS:



SOBRE EL JUEGO

Gems Seeker es un divertido y colorido videojuego diseñado especialmente para niños. El protagonista es un simpático pico minero que tiene una misión muy importante: buscar y abrir los 5 cofres mágicos escondidos en el mapa. Para lograrlo, el jugador deberá usar el poder de las gemas:

- Si no tiene la gema correcta, deberá recoger la más cercana en número (sucesor o predecesor).
- Cada gema es única y representa un poder especial que ayudará al pico en su aventura.

El objetivo final es abrir todos los cofres y llegar al portal de salida, ¡todo esto sin ser atrapado por el enemigo!

CARACTERÍSTICAS PRINCIPALES

- 🎮 **Jugabilidad sencilla y amigable para niños.**
- 💎 **Recolección de gemas con mecánicas de sucesor y predecesor.**
- 🔑 **5 cofres mágicos que deben abrirse para completar la misión.**
- 🚪 **Portal final que marca la victoria.**
- ⚔️ **Desafío: evitar al enemigo mientras se busca el tesoro.**

◆ CLASES ◆



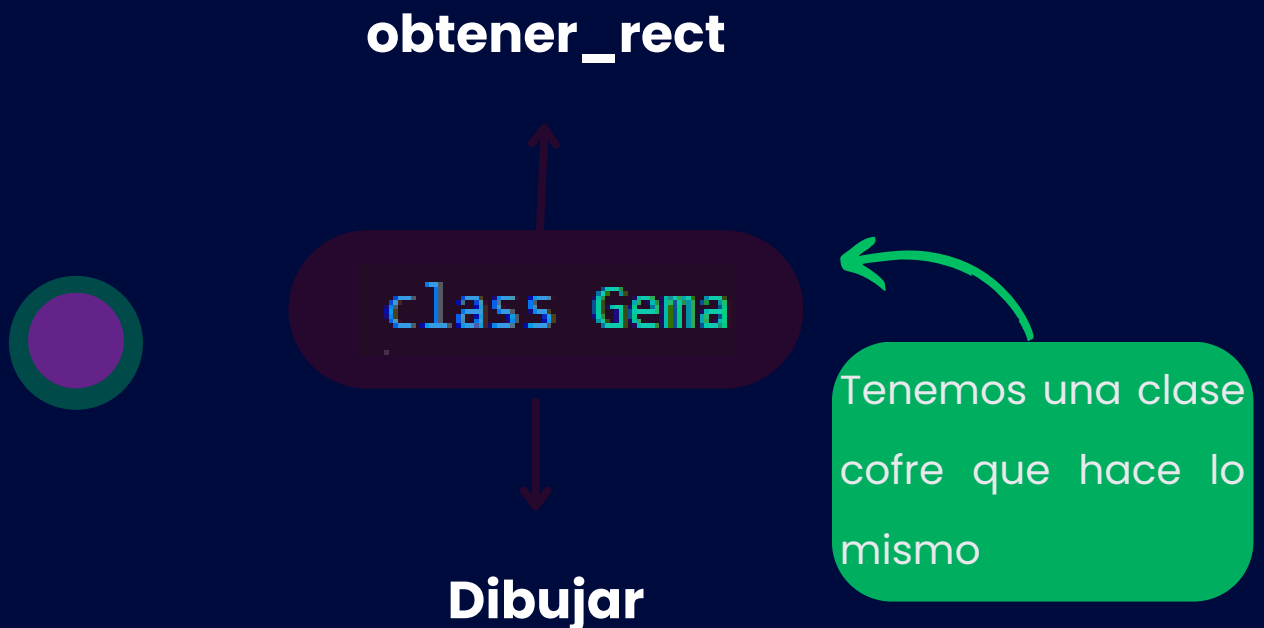
La clase `ArbolBST` implementa un Árbol Binario de Búsqueda que gestiona las gemas del juego. Permite insertar, buscar y eliminar nodos (gemas), manejando correctamente los casos de eliminación con cero, uno o dos hijos. Además, incluye operaciones especiales para obtener el mínimo, máximo, sucesor y predecesor, y soporta recorridos inorden, preorden y postorden para mostrar el inventario, guardar el estado o reiniciar el juego. Esta estructura asegura que las gemas se almacenen y consulten de forma eficiente, integrando la lógica de datos con la mecánica del juego.

Mover

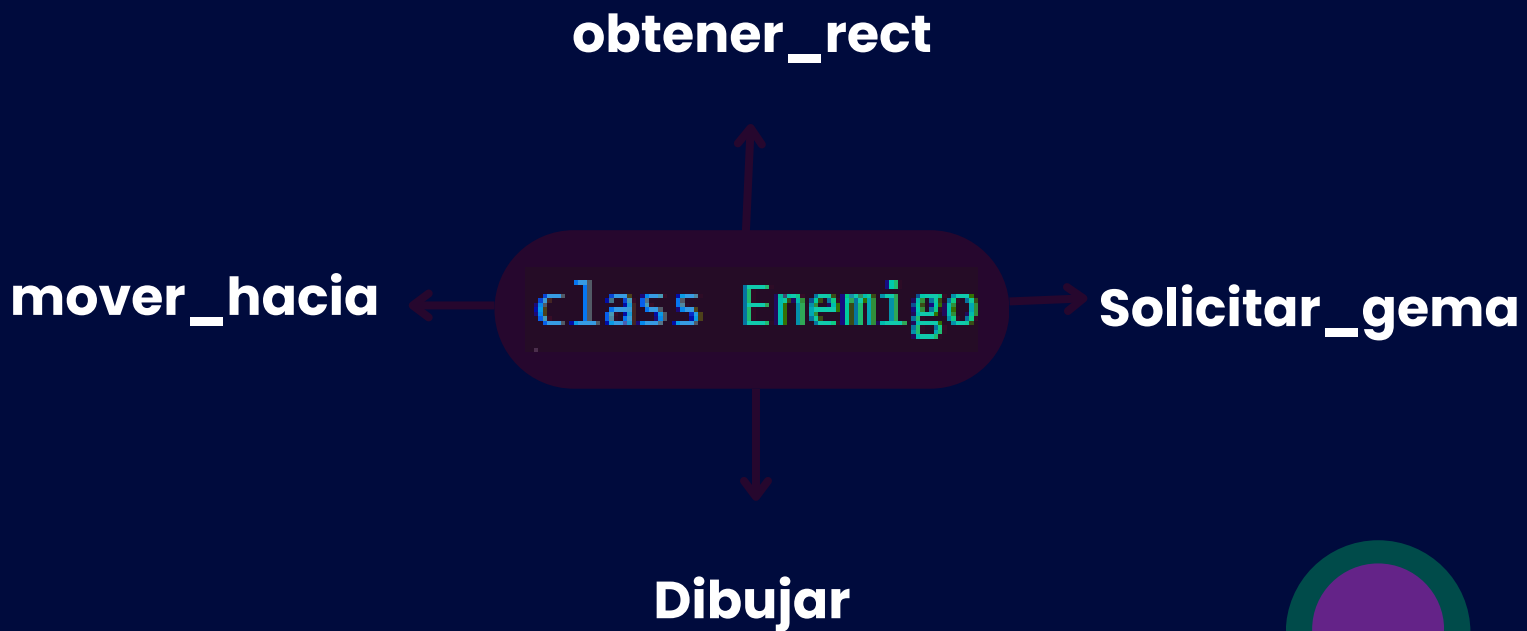
`class Jugador`

Dibujar

La clase Jugador representa al personaje principal dentro del juego. Define su posición y tamaño mediante un rectángulo de Pygame, su velocidad de movimiento, y un inventario basado en el Árbol Binario de Búsqueda para almacenar las gemas recolectadas. Incluye métodos para moverse dentro de los límites del escenario y para dibujarse en pantalla, asegurando así la interacción del jugador con el entorno y la integración directa con la lógica del árbol.

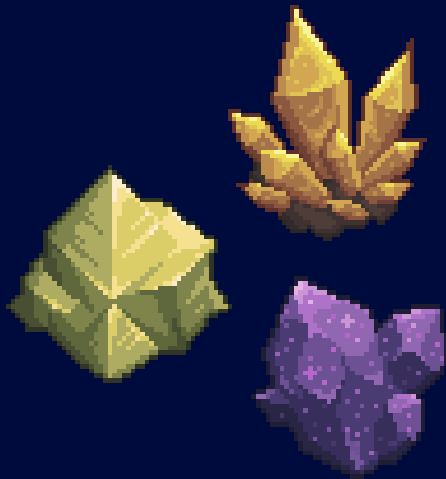


La clase `Gema` modela los objetos que el jugador puede recolectar en el juego. Cada gema se crea con un poder aleatorio entre 1 y 100, un nombre asociado y una posición (x, y) en el mapa. Además, cuenta con un radio que define su tamaño y un método para dibujarse como un círculo en pantalla, mostrando en su interior el valor de poder. También incluye un método para obtener su rectángulo delimitador, lo que permite detectar colisiones con el jugador y así gestionar su recolección e inserción en el inventario.



La clase `Enemigo` representa a los rivales que persiguen al jugador en el juego. Se definen por su posición (x, y) , un rectángulo delimitador y una velocidad con la que se mueven hacia la ubicación del jugador, implementada en el método `mover_hacia`. Visualmente, se dibujan en pantalla como un rectángulo. Además, esta clase introduce la mecánica de desafío mediante el método `solicitar_gema`, donde el enemigo pide una gema aleatoria: si el jugador la tiene, se elimina del inventario; si no, se busca un sucesor o predecesor en el árbol, y en caso de no existir, se elimina la raíz. Así, los enemigos no solo agregan dinámica al movimiento, sino que también ponen a prueba la lógica del Árbol Binario de Búsqueda dentro del juego.

◆ EVENTOS ◆



GEMAS



```
def generar_gemas(self, cantidad):
```

```
def evento_gema_especial(self):
```

Los métodos `evento_gema_especial` y `generar_gemas` controlan la aparición de gemas en el mundo del juego. El primero introduce gemas únicas y poderosas (como la Gema del Fuego o la Gema del Agua) con altos niveles de poder, colocándolas aleatoriamente en el mapa y notificando al jugador. El segundo se encarga de generar varias gemas comunes validando que tengan poderes distintos, evitando repeticiones para mantener la diversidad. Ambos métodos enriquecen la jugabilidad y se integran con el Árbol Binario de Búsqueda al ampliar el inventario dinámico de gemas.



ENEMIGO



```
def generar_enemigo(self):
```

Este metodo crea enemigos en posiciones seguras, los mueve hacia el jugador y detecta colisiones. Si atrapan al jugador, solicitan una gema de su inventario (que reaparece en otra ubicación) o muestran un mensaje si no hay gemas, eliminando al enemigo tras el encuentro.



PORTAL



```
def intentar_usar_portal(self, portal)
```

```
def generar_portal(self)
```

Los métodos intentar_usar_portal y generar_portal permiten la interacción con portales mágicos: el primero consume la gema de mayor poder del inventario para cruzar y desactivar el portal, mientras que el segundo crea un portal en una posición aleatoria del mapa.



COFRES



```
def intentar_abrir_cofre(self, cofre):
```

```
def generar_cofres(self, cantidad):
```

controlan la mecánica de los cofres en el juego. El primero coloca cofres en posiciones aleatorias del mapa, lejos del jugador inicial. El segundo gestiona su apertura: si el jugador tiene la gema exacta requerida, el cofre se abre; en caso contrario, se busca la gema más cercana en el inventario para intentarlo. Si no hay gemas disponibles, el cofre permanece cerrado y se muestra un mensaje en pantalla.



TRAMPA



```
def evento_trampa(self):
```

El método `evento_trampa` activa una trampa aleatoria que afecta al jugador. Si el inventario está vacío, se muestra un mensaje indicando que la trampa fue esquivada. En caso contrario, selecciona al azar una gema del inventario, la elimina y notifica al jugador que la perdió.



VIDEOJUEGO



Conclusion

El desarrollo del juego **Gems Seeker** permitió cumplir con los objetivos planteados en torno al uso de estructuras de datos. Se implementó desde cero un Árbol Binario de Búsqueda, incorporando correctamente las operaciones de inserción, búsqueda y eliminación. Asimismo, se aplicaron los recorridos inorden, preorden y postorden, cada uno vinculado a funcionalidades concretas dentro de la dinámica del juego. También se integraron las operaciones especiales de mínimo, máximo, predecesor y sucesor, útiles en la interacción con cofres, enemigos y trampas. Finalmente, el BST se consolidó como el núcleo de la mecánica, ya que el inventario de gemas y la resolución de eventos dependen directamente de su correcta gestión, logrando así unir teoría y práctica en una experiencia lúdica y formativa.

